

PART IV — FROM RAG TO AGENTIC RAG

Chapter 18

Implementing the Agent with `llm.invoke(tools=...)`

“A loop is not an agent until something inside it can decide the loop is done.”

Chapter 18 — Implementing the Agent with `llm.invoke(tools=...)`

Nothing here is abstract. Every mechanism in this chapter exists in Memora's `run_agent` today, in the form a real failure forced it into: a synthetic tool call injected when the model skips a required step, a message scrubbed rather than deleted so a chat protocol invariant survives compression, a sidecar message that keeps a validator's opinion visible without letting it masquerade as retrieved evidence, and an exit condition that leaves room for exactly one more round-trip before giving up.

18.1 `response.content` vs `response.tool_calls` — the two shapes of a reply

Definition — Tool call

A structured request attached to a model's response — a name and an argument dictionary — that the orchestrator, not the model, is responsible for executing and returning a result for.

Every `llm_invoke` call in the RETRIEVE phase returns a response with two fields worth reading separately: `.content`, the model's plain text, and `.tool_calls`, a list of structured requests. A response is never partially one or the other in practice — the orchestrator's entire branching logic in this phase hinges on which one arrived.

```
resp_content      = str(getattr(response, "content", ""))
resp_tool_calls   = getattr(response, "tool_calls", []) or []

if not resp_tool_calls:
    # LLM emitted text with no tool calls — treat as "done retrieving"
    phase = "COMPRESS"
    continue
```

Notice what does *not* happen in the empty-tool-calls branch: the bare text response is never appended to `messages`. Appending it would put an unstructured assistant utterance into a transcript the rest of the loop expects to be tool-call-shaped, for no benefit — the text itself is discarded, only the *signal* that the model considers retrieval finished is kept, by moving the phase forward.

18.2 Executing tool calls in your loop

A response can carry more than one tool call in a single turn. `run_agent` deduplicates identical `retrieve_documents` queries within the same batch before executing anything — cheap insurance against a model asking the same question twice in one breath — then dispatches each surviving call by name.

```
for tool_call in deduped:
    name, args = tool_call["name"], tool_call["args"]

    if name == "retrieve_documents":
        result = _handle_retrieve_documents_call(tool_call)
    elif name == "compress_context":
        result = callables["compress_context"]()
        compress_called_this_iter = True
    else:
        result = f"Unknown tool '{name}'."    # never silently ignored
```

The `else` branch matters as much as the two real ones. A model that hallucinates a tool name — or calls one from an earlier prompt version still lingering in its context — gets an explicit, visible rejection message fed back as a real tool result, not a silent no-op. The loop stays auditable even when the model asks for something that does not exist.

18.3 Feeding tool results back as role: "tool" messages

A chat-completions API enforces a strict pairing: every `tool\_calls` entry in an assistant message needs exactly one corresponding message with `role: "tool"` and a matching `tool\_call\_id`, before the next assistant turn is valid. `run\_agent` appends the assistant response first, then one tool message per executed call, in the same order.

```
messages.append(response)    # the assistant turn, tool_calls and all

for tool_call in deduped:
    result = dispatch(tool_call)
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call["id"],
        "content": result,
    })
```

Get this ordering or pairing wrong — skip a call, append results out of order, reuse an id — and the next `llm\_invoke` call does not fail gracefully. It fails at the provider, with a protocol-level rejection that has nothing to do with the model's reasoning and everything to do with the shape of the conversation you sent it.

18.4 Injecting real retrieved context into quality checks

`check\_answer\_quality` was not always the plain function Chapter 17 described. An earlier version exposed it as a callable tool, and the 8B model exploited the freedom exactly as such freedom tends to get exploited: it called the quality check in iteration one, before any retrieval had happened, grading an answer it had just fabricated from training knowledge rather than retrieved chunks. ADR-020's fix removed it from the tool schema entirely and made the orchestrator call it directly in the JUDGE phase, with the real compressed context as an explicit argument — not whatever the model claimed the context was.

Common pitfall — Mutating a tool call's args dict in place

A tool call's `args` dictionary is the same Python object already sitting inside the assistant message appended to `messages`. Overwriting one of its keys in place — to inject real context in place of what the model supplied, for instance — silently rewrites the conversation history: the transcript now shows the model having "asked for" data it never actually requested. Build a new dictionary for the call you actually execute (`\*\*tool\_call["args"], "context": real\_context`) and leave the object living inside `messages` untouched. History should record what happened, not what the orchestrator wishes had happened.

This is the same context-injection principle Chapter 17.3 introduced — keep server-owned data out of the model's hands — carried one step further: injection has to happen without corrupting the very record the next LLM call will read back as its own prior turn.

18.5 Exit conditions and safeguarding against infinite loops

Three independent budgets bound `run_agent`, and none of them alone is sufficient — each caps a different resource the loop could otherwise exhaust.

Budget	Bounds	Caught by
<code>'MAX_ITERATIONS'</code>	Total passes through the state machine	<code>'while iterations < MAX_ITERATIONS'</code>
<code>'MAX_TOTAL_RETRIEVALS'</code>	Retrieval calls across the whole run	Checked inside <code>'_handle_retrieve_documents_call'</code>
<code>'MAX_TOOL_CALLS_PER_ITERATION'</code>	Tool calls accepted from one LLM turn	Slice applied before dispatch: <code>'resp_tool_calls[:MAX_TOOL_CALLS_PER_ITERATION]'</code>

The JUDGE phase adds a fourth, subtler condition on top of the raw counters: `'budget_ok'` requires `'iterations < MAX_ITERATIONS - 1'`, not simply `'< MAX_ITERATIONS'`. Looping back to RETRIEVE only to discover on the very next iteration that the budget is now exhausted would waste an entire iteration on a retry the loop could never finish — reserving one iteration's headroom guarantees a retry, once granted, always has room to reach a final answer.

When every budget is exhausted and no phase has returned, the loop falls through to a final, unconditional check: return the best draft available, or an honest "max iterations reached" message if there was never a draft to fall back on. An agent loop without an unreachable-fallthrough case is a loop that can, on a bad enough day, simply never return.

18.6 Writing `agent_query.py`

`run_agent` takes every dependency — the LLM clients, the tool schemas and callables, the quality-check function, the retriever, a shared mutable `'agent_state'` dict — as an explicit argument rather than a module-level global. Nothing about the function reaches outside itself for state, which is what makes the whole loop testable with fakes standing in for a real model and a real vector store.

```
def run_agent(
    query: str, llm, merge_llm, judge_llm,
    tool_schemas: list, callables: dict,
    check_answer_quality,          # plain callable, NOT an LLM tool
    retriever: RAGRetriever, agent_state: dict,
    blocked_variants: list[str] | None = None,
    prior_thumbsdowns: list[dict] | None = None,
) -> tuple[dict, list[str]]:
    ...
```

The return type is worth pausing on: a tuple of the answer payload *and* a list of newly-failed query variants. The second element is not an afterthought — it is how this run's failures become tomorrow's blocked variants (Chapter 14's `'_ROLE_AND_RULES'` injection), a self-learning feedback loop that only works because the loop reports its own failures honestly, not just its successes.

18.7 From open loop to phase state machine

An earlier version of this agent was a free-form loop: call the model, execute whatever it asked for, repeat, with the model itself deciding what stage of the process it was in. ADR-020's fix was not a patch on that design — it replaced the design. `run_agent` tracks its own `phase` variable explicitly, and the model's freedom is scoped to what a given phase permits, never to the sequence of phases itself.

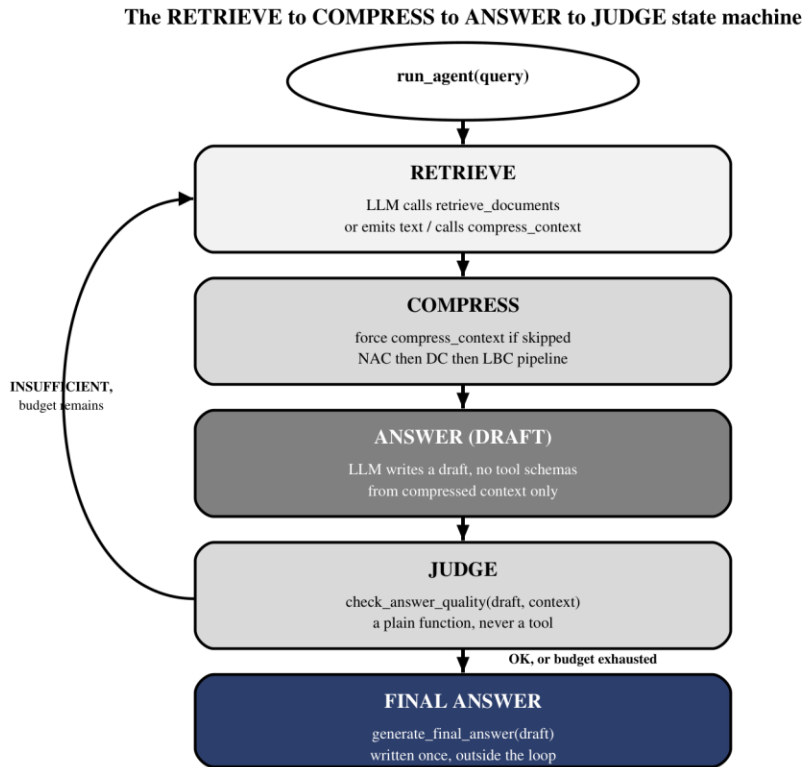


Figure 18.1 — Four phases, one loop-back edge, and exactly one way out.

Phase	Model's freedom	Orchestrator's guarantee
RETRIEVE	Choose queries, decide when done	Retrieval and total-call caps always enforced
COMPRESS	None — model is not consulted	<code>compress_context</code> runs exactly once, forced if skipped
ANSWER	Write a draft from given context	No tool schemas offered — text only
JUDGE	None — a plain function decides	Verdict always computed the same way

Figure 18.1's single loop-back edge is deliberate: JUDGE can send the run back to RETRIEVE, and nothing else can. Every other transition moves strictly forward. A state machine with one controlled cycle is auditable in a way a graph with cycles between every phase never could be — you can always answer "how many times can this run repeat itself, and under exactly what condition?"

18.8 Synthetic-injection — when the LLM skips a required phase

Two real gaps in what the model reliably does on its own are closed the same way: the orchestrator constructs the tool call the model should have made, appends it to `messages` as if the model had made it, executes it, and appends the result — all before the model gets another turn.

```
def _inject_synthetic_tool_call(tool_name: str, tool_args: dict, reason: str) -> str:
    synth_id = f"call_sys_{tool_name}_{uuid.uuid4().hex[:12]}"
    messages.append({
        "role": "assistant",
        "content": f"[system-injected tool call] Running {tool_name}()",
        "tool_calls": [{"id": synth_id, "name": tool_name, "args": dict(tool_args)}],
    })
    result_str = callables[tool_name](**tool_args)
    messages.append({"role": "tool", "tool_call_id": synth_id, "content": result_str})
    return result_str
```

18.8.1 Retrieval with sub-query generation disabled

With `ENABLE\_SUB\_QUERY\_GENERATION` off, the model is never even asked to choose a query — the orchestrator injects a `retrieve\_documents` call with the original question verbatim before the first LLM call of the run, and moves straight to COMPRESS. The model's turn begins one phase later than it otherwise would.

18.8.2 Compression skipped at the end of RETRIEVE

The far more common case: the model stops calling `retrieve\_documents` — either by emitting plain text or exhausting its retrieval budget — without ever calling `compress\_context` itself. The COMPRESS phase checks `agent\_state["compress\_done"]` and, if it is still `False`, injects the call before the model is consulted again. The model experiences this as compression having simply already happened.

18.8.3 The user-role nudge after a bad verdict

One more pattern belongs in this family even though it injects a message rather than a tool call: when JUDGE returns INSUFFICIENT with budget remaining, the orchestrator appends a `role: "user"` message explaining exactly why the draft failed and instructing the model to try genuinely different query angles, then resets `compress\_done` and loops back to RETRIEVE. It is the same idea as a synthetic tool call — the orchestrator, not the model, decides what happens next — expressed as a steering message instead of a fabricated action, because what is missing here is not a skipped step but a bad result the model needs a clean, specific reason to try again.

18.9 Message-list scrubbing without breaking the assistant to tool_call_id pairing

Once `compress\_context` runs, the raw chunks a `retrieve\_documents` call returned earlier in the conversation are redundant — their compressed replacement is now the tool's own result — and expensive to keep paying prompt-token cost for on every subsequent turn. The fix is not to delete those earlier messages.

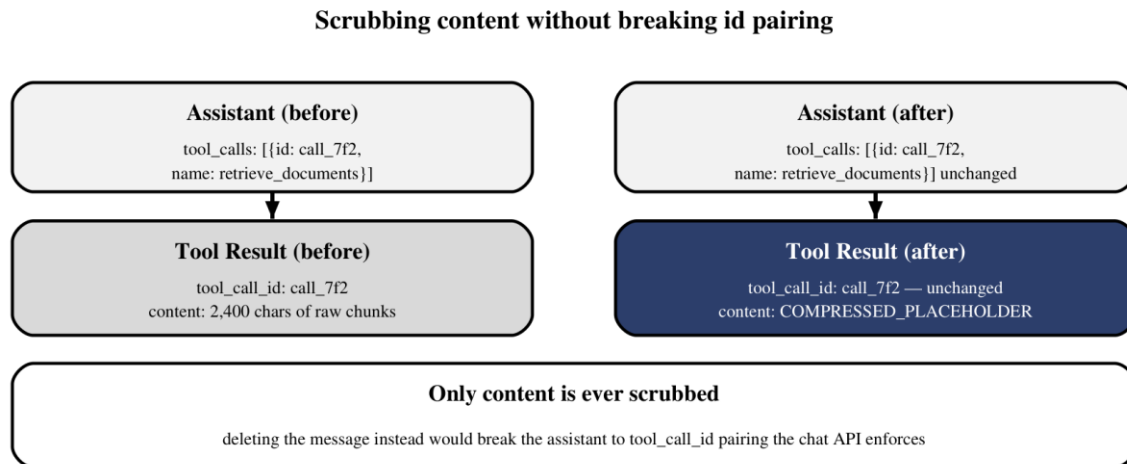


Figure 18.2 — The tool_call_id never changes; only the content behind it does.

Deleting a `retrieve\_documents` tool-result message would leave its parent assistant message's `tool\_calls` entry pointing at an id with no matching result — the exact protocol violation Section 18.3 warned about, self-inflicted this time by cleanup code instead of a dispatch bug. The fix in Figure 18.2 mutates only `content`, replacing it with a short, constant placeholder, and leaves every id exactly where the model left it.

```
for m in messages:
    if m.get("role") != "tool":
        continue
    origin = tool_call_id_to_name.get(m.get("tool_call_id", ""))
    if origin == "retrieve_documents" and m["content"] != COMPRESSED_PLACEHOLDER:
        m["content"] = COMPRESSED_PLACEHOLDER # never delete the message itself
```

18.10 The `_judge` sidecar message

Retrieval validation (Chapter 12's researched-but-not-shipped judge, wired in here) needs to tell the model something happened without pretending its verdict was retrieved evidence. `run_agent` appends the verdict as its own tool message, keyed off the *same* `tool_call_id` as the retrieval it is judging, with a `_judge` suffix appended.

```
messages.append({
    "role": "tool",
    "tool_call_id": tool_call["id"] + "_judge",
    "content": f"[RETRIEVAL JUDGE] {verdict_summary}",
})
```

The suffix is not decorative — the scrubbing logic in Section 18.9 explicitly strips it back off (`tcid[:-len("_judge")]`) to look up which tool the sidecar belongs to, so a judge verdict about a `retrieve_documents` call is correctly identified as commentary *about* retrieved evidence, never mistaken for retrieved evidence itself, while still living in the transcript at exactly the point the model needs to see it.

18.11 Token-usage accounting per iteration

Every `llm_invoke` response carries a `token_usage` dictionary in `response_metadata`, and `run_agent` reads it after every single call — RETRIEVE, DRAFT, and FINAL alike — accumulating into two running totals that survive the entire run, not just the current phase.

```
usage = (getattr(response, "response_metadata", {}) or
{}).get("token_usage", {})
total_prompt_tokens += usage.get("prompt_tokens", 0)
total_completion_tokens += usage.get("completion_tokens", 0)
```

Both totals ride along in every return payload the function produces, win or lose. A caller — a benchmark script, a cost dashboard, the `Execution Time Comparison.md` workflow Chapter 13B's provider decisions leaned on — never has to reconstruct token spend from a debug log after the fact. The loop that spent the tokens is the loop that reports them, at the moment it has the number, because a total computed later from scattered log lines is a total someone eventually gets wrong. `run_agent` is a complete, working agent — and, deliberately, the last version of this architecture built by hand, message list and all.